



RAMAIAH

College of Arts, Science & Commerce-Autonomous

www.msrmcas.edu.in

IV Semester

Database Management System

LABORATORY

ACADEMIC YEAR

2024 – 2025

LABORATORY

MANUAL

Department of Computer Applications



RAMAIAH

College of Arts, Science & Commerce-Autonomous

www.msrmcasc.edu.in

Department of BCA

IV Semester

Database Management System Lab

ACADEMIC YEAR 2024 – 2025

**NAME OF THE
STUDENT** :

**UNIVERSITY SEAT
No.** :

BATCH :



RAMAIAH
College of Arts, Science &
Commerce- Autonomous

Department of BCA

LABORATORY CERTIFICATE

*This is to certify that Mr. /Ms. _____ bearing USN _____ of
_____ semester and _____ section has satisfactorily completed the course of
experiments in **Problem Solving Technique Using C** prescribed by the
BENGALURU CITY UNIVERSITY of this Institute for the academic year
2024- 2025 .*

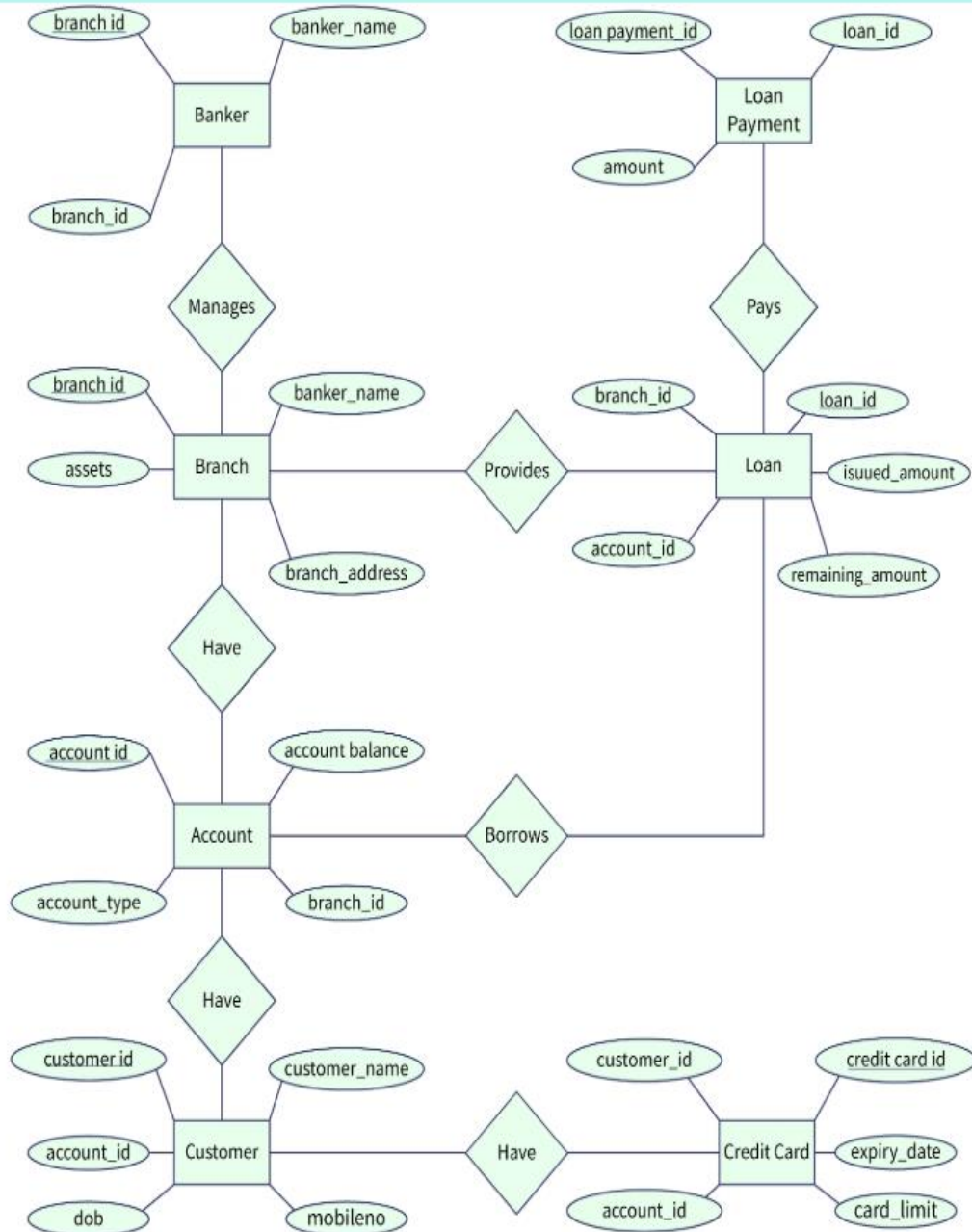
MARKS	
Maximum Marks	Marks Obtained

1. Case Study: Entity-Relationship (E-R) Diagram and Database Schema Design for Banking System

Objective: To design an E-R diagram for banking system and convert the entities and relationships into relational tables. This exercise helps in understanding the process of database modeling from conceptual design to physical implementation.

In this ER diagram of the bank database, we have eight entities,

1. Customer, To represent the customers.
2. Banker, To represent the Banker, who manages the entire branch.
3. Branch, To represent a branch of a bank.
4. Loan, To represent the loan granted by the branch to the customer's account.
5. Account, To represent the bank account of any customer.
6. Transaction, To represent the transactions of customers for any account.
7. Credit Card, To represent the Credit card of any associated customer and account.
8. Loan Payment, To represent the Payment towards the loan.



2. Case Study: Social Media System Objective: To design and implement a database schema for a social media system. This involves various entities and relationships within the system and converting this model into relational tables, data manipulation and queries to retrieve and analyze data from the system.

The SOCIAL MEDIA database has a table with the following attributes.
The primary key is underlined.

USERS(user_id: int, username: string, email: string, dob: date)

- a) Create the above table.
- b) Remove the existing attribute email from the table.
- c) Add a new attribute phone_no to the existing table.
- d) Enter five tuples into the table.
- e) Display all the tuples in the USERS table.
- f) Create a POSTS table with post_id, user_id (foreign key), and content.
- g) Enter five tuples into the post table.
- h) Retrieve the usernames and their posts from the database.
- i) Display all the tuples in the USERS and POSTS table.

- a) Create the above table.

```
CREATE TABLE USERS (  
  user_id INT PRIMARY KEY,  
  username VARCHAR(50),  
  email VARCHAR(100),  
  dob DATE  
);
```

- b) Remove the existing attribute email from the table.

```
ALTER TABLE USERS DROP COLUMN email;
```

- c) Add a new attribute phone_no to the existing table.

```
ALTER TABLE USERS ADD phone_no VARCHAR(15);
```

d) Enter five tuples into the table.

```
INSERT INTO USERS (user_id, username, dob, phone_no) VALUES  
(101, 'Alice', TO_DATE('2000-05-12','YYYY-MM-DD'), '9876543210');
```

```
INSERT INTO USERS VALUES  
(102, 'Bob', TO_DATE('1998-08-25','YYYY-MM-DD'), '9876543211');
```

```
INSERT INTO USERS VALUES  
(103, 'Charlie', TO_DATE('2002-02-15','YYYY-MM-DD'), '9876543212');
```

```
INSERT INTO USERS VALUES  
(104, 'David', TO_DATE('1999-11-05','YYYY-MM-DD'), '9876543213');
```

```
INSERT INTO USERS VALUES  
(105, 'Emma', TO_DATE('2001-09-30','YYYY-MM-DD'), '9876543214');
```

e) Display all the tuples in the USERS table.

```
SELECT * FROM USERS;
```

f) Create a POSTS table with post_id, user_id (foreign key), and content.

```
CREATE TABLE POSTS (  
  post_id INT PRIMARY KEY,  
  user_id VARCHAR(20),  
  content TEXT NOT NULL,  
  FOREIGN KEY (user_id) REFERENCES USERS(user_id)  
);
```

g) Enter five tuples into the post table.

```
INSERT INTO POSTS (post_id, user_id, content) VALUES  
(1, 101, 'Alice is posting for the first time!');
```

```
INSERT INTO POSTS VALUES  
(2, 102, 'Bob shares his thoughts on technology.');
```

```
INSERT INTO POSTS VALUES
```

```
(3, 103, 'Charlie writes about his travel experiences.');
```

```
INSERT INTO POSTS VALUES
```

```
(4, 104, 'David shares his opinion on politics.');
```

```
INSERT INTO POSTS VALUES
```

```
(5, 105, 'Emma writes a blog on fitness and health.');
```

h) Retrieve the usernames and their posts from the database.

```
SELECT USERS.username, POSTS.content
```

```
FROM USERS
```

```
JOIN POSTS ON USERS.user_id = POSTS.user_id;
```

i) Display all the tuples in the USERS and POSTS table.

```
SELECT * FROM USERS;
```

```
SELECT * FROM POSTS;
```

3. Case Study: Database Schema for a Customer-Sale Scenario Objective:

To design and implement a database schema to manage customer sales, perform various queries, and derive meaningful insights from the data. Consider the customer-sales database given below. The primary keys are underlined and the datatypes are specified.

Customer(Customer id:int,name:string,address:string)

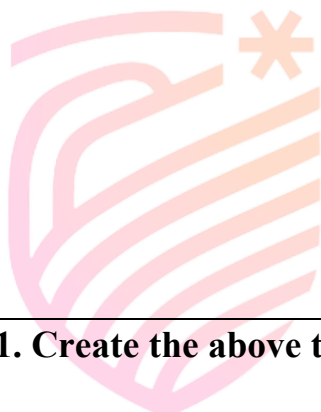
Product(Product _id:int,name:string,price:float)

Sales(sale _id:int,customer _id:int,sale_date:date)

Sales details(sale _id:int,product _id:int,quantity:int)

1) Create the above table

- 2) Enter five tuples into the table
- 3) Display all the tuples in the customer table
- 4) Find total sales for each customer
- 5) List all products purchased by a specific customer (Ex: Alice)
- 6) Find the most sold product
- 7) Find the customer who spent the most



RAMAIAH
College of Arts, Science

1. Create the above table

```
CREATE TABLE CUSTOMER (  
customer_id INT PRIMARY KEY,  
name VARCHAR(255),  
address VARCHAR(255)  
);
```

```
CREATE TABLE PRODUCT (  
product_id INT PRIMARY KEY,  
name VARCHAR(255),  
price FLOAT  
);
```

```
CREATE TABLE SALE (  
sale_id NUMBER PRIMARY KEY,  
customer_id NUMBER,  
sale_date DATE,  
CONSTRAINT fk_sale_customer  
FOREIGN KEY (customer_id)
```

```
REFERENCES CUSTOMER(customer_id)
);

CREATE TABLE SALE_DETAILS (
sale_id INT,
product_id INT,
quantity INT,
PRIMARY KEY (sale_id, product_id),
FOREIGN KEY (sale_id) REFERENCES SALE(sale_id),
FOREIGN KEY (product_id) REFERENCES PRODUCT(product_id)
);
```

2. Enter five tuples into the table

```
INSERT INTO CUSTOMER VALUES (1, 'Alice', 'New York');
INSERT INTO CUSTOMER VALUES (2, 'Bob', 'Los Angeles');
INSERT INTO CUSTOMER VALUES (3, 'Charlie', 'Chicago');
```

```
INSERT INTO PRODUCT VALUES (101, 'Laptop', 1000);
INSERT INTO PRODUCT VALUES (102, 'Phone', 500);
INSERT INTO PRODUCT VALUES (103, 'Tablet', 300);
```

```
INSERT INTO SALE VALUES
(201, 1, TO_DATE('2025-01-15', 'YYYY-MM-DD'));
INSERT INTO SALE VALUES
(202, 2, TO_DATE('2025-01-16', 'YYYY-MM-DD'));
```

```
INSERT INTO SALE_DETAILS VALUES (201, 101, 1);
INSERT INTO SALE_DETAILS VALUES (201, 102, 2);
INSERT INTO SALE_DETAILS VALUES (202, 103, 1);
```

3. Display all the tuples in the customer table

```
SELECT * FROM CUSTOMER
```

4. Find Total Sales for Each Customer

```
SELECT C.name, SUM(P.price * SD.quantity) AS total_spent
FROM CUSTOMER C, SALE S, SALE_DETAILS SD, PRODUCT P
WHERE C.customer_id = S.customer_id
AND S.sale_id = SD.sale_id
AND SD.product_id = P.product_id
GROUP BY C.name;
```

5. List All Products Purchased by a Specific Customer (Example: Alice)

```
SELECT P.name, SD.quantity
FROM CUSTOMER C, SALE S, SALE_DETAILS SD, PRODUCT P
WHERE C.customer_id = S.customer_id
AND S.sale_id = SD.sale_id
AND SD.product_id = P.product_id
AND C.name = 'Alice';
```

6. Find the Most Sold Product

```
SELECT name, total_sold
FROM ( SELECT P.name, SUM(SD.quantity) AS total_sold
FROM PRODUCT P, SALE_DETAILS SD WHERE P.product_id =
SD.product_id GROUP BY P.name ORDER BY total_sold DESC )
WHERE ROWNUM = 1;
```

7. Find the Customer Who Spent the Most

```
SELECT name, total_spent
FROM (
    SELECT C.name, SUM(P.price * SD.quantity) AS total_spent
    FROM CUSTOMER C, SALE S, SALE_DETAILS SD, PRODUCT P
```

```
WHERE C.customer_id = S.customer_id
AND S.sale_id = SD.sale_id
AND SD.product_id = P.product_id
GROUP BY C.name
ORDER BY total_spent DESC
)
WHERE ROWNUM = 1;
```

4. Case Study: Designing a Database Schema for an Employee

Management System Objective: Explore the design of a database schema for an Employee Management System (EMS) that supports the management of employee information, departments, salaries, and the relationships between them.

- Create the table employee.
- Enter five tuples into the table.
- Display all the tuples in the Employee table.
- Display employee number and their salary
- Find the sum and average salaries of employees of a particular department
- Display employee information in descending order of their date of joining with employee name
- Find the highest salary that an employee draws
- Find the details of employees whose name is jhon and salary is greater than 30,0000
- Find the least salary that an employee draw

- a). Create a table called employeee with the specified columns (empnumber, empname, salary, date_of_joining, department, phone_number)

```
CREATE TABLE employeee (
    empnumber INT PRIMARY KEY , -- Employee number
    empname VARCHAR(100) ,      -- Employee name
    salary DECIMAL(10, 2) ,      -- Employee salary
    date_of_joining DATE,        -- Date of joining
    department VARCHAR(100) ,    -- Department
    phone_number VARCHAR(15)     -- Employee phone number
);
```

b) Enter five tuples into the table.

```
INSERT INTO employeee VALUES (1, 'John Doe', 60000,  
TO_DATE('2015-06-15','YYYY-MM-DD'), 'IT', '123-456-7890');
```

```
INSERT INTO employeee VALUES (2, 'Jane Smith', 75000,  
TO_DATE('2018-03-10','YYYY-MM-DD'), 'Finance', '987-654-3210');
```

```
INSERT INTO employeee VALUES (3, 'Mark Johnson', 85000,  
TO_DATE('2012-08-25','YYYY-MM-DD'), 'IT', '555-123-4567');
```

```
INSERT INTO employeee VALUES (4, 'Emily Davis', 90000,  
TO_DATE('2016-05-30','YYYY-MM-DD'), 'Marketing', '555-234-5678');
```

```
INSERT INTO employeee VALUES (5, 'David Brown', 95000,  
TO_DATE('2014-02-19','YYYY-MM-DD'), 'Sales', '555-345-6789');
```

c) Display all the tuples in the Employee table.

```
SELECT * FROM employeee;
```

d) Display Employee Number and Their Salary.

```
SELECT empnumber, salary FROM employeee;
```

e) Find the Sum and Average Salaries of Employees of a Particular Department.

```
SELECT department,  
SUM(salary) AS total_salary,  
AVG(salary) AS average_salary
```

```
FROM employeee  
WHERE department = 'IT'  
GROUP BY department;
```

f) Display Employee Information in Descending Order of Their Date of Joining with Employee Name.

```
SELECT empname, date_of_joining  
FROM employeee  
ORDER BY date_of_joining DESC;
```

g) Find the Highest Salary That an Employee Draws

```
SELECT MAX(salary) AS highest_salary FROM employeee;
```

h) Find the details of employees whose name is John and salary is greater than 30,000

```
SELECT empnumber, empname, salary, date_of_joining, department,  
phone_number
```

```
FROM employeee
```

```
WHERE empname LIKE 'John%'
```

```
AND salary > 30000;
```

i) Find the least salary that an employee draw

```
SELECT MIN(salary) AS least_salary FROM employeee;
```

j) Find the Details of Employees Whose Name is 'John' and Salary is Greater Than 300,000

```
SELECT empnumber, empname, salary, date_of_joining, department,  
phone number
```

```
FROM employee WHERE empname = 'John' AND salary > 300000;
```

5. Case Study: Student Library Database Management System

Objective: To design and implement a database schema for managing a student library. This includes tracking students, memberships, books, and issue records, as well as performing various SQL operations to extract useful information.

Create table Student with following attributes (student_id as primary key, first name varchar(50), last name- varchar(50), DOB - Date, Email- varchar(100))

Create table Membership with following attributes (membership_id INT primary key, student_id INT Foreign key, start_date- Date, end_date- Date)

Create table Publishers with following attributes (publisher_id INT PRIMARY KEY, publisher_name varchar (100))

Create table books with following attributes (book_id INT PRIMARY KEY, title varchar (100), author varchar (100), price DECIMAL (10, 2), publisher_id INT, FOREIGN KEY (publisher_id) REFERENCES Publishers (publisher_id))

Create table Book_issues with following attributes (issue_id INT PRIMARY KEY, student_id INT, book_id INT, issue_date DATE, return_date DATE, FOREIGN KEY (student_id) REFERENCES Students(student_id), FOREIGN KEY (book_id) REFERENCES Books(book_id))

- a) Create the above tables.
- b) Enter 5 tuples into the tables.
- c) Display Different Publishers from the List.
- d) List Books Whose Price Range is Between 100 and 300.
- e) List Books Published by a Specific Publisher.
- f) List All Book Titles with the Corresponding Student's First and Last Names and the Issue Date.
- g) List All Memberships with Student Name, Start Date, and End Date.
- h) Find Students Who Have Not Returned Books After the Due Date.



a) Create the above tables

- i) Create the student table.

```
CREATE TABLE Students (  
    student_id Number PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    dob DATE,  
    email VARCHAR(100)  
);
```

- ii) Create the Memberships table

```
CREATE TABLE Memberships (  
    membership_id Number PRIMARY KEY,  
    student_id INT,  
    start_date DATE,  
    end_date DATE,  
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
```



```
);
```

iii) Create the Publishers table

```
CREATE TABLE Publishers (  
    publisher_id Number PRIMARY KEY,  
    publisher_name VARCHAR(100)  
);
```

iv) Create the Books table

```
CREATE TABLE Books (  
    book_id Number PRIMARY KEY,  
    title VARCHAR(100),  
    author VARCHAR(100),  
    price DECIMAL(10, 2),  
    publisher_id INT,  
    FOREIGN KEY (publisher_id) REFERENCES Publishers(publisher_id)  
);
```

v) Create the Books issue table

```
CREATE TABLE Book_Issues (  
    issue_id Number PRIMARY KEY,  
    student_id Number,  
    book_id Number,  
    issue_date DATE,  
    return_date DATE,  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (book_id) REFERENCES Books(book_id)  
);
```

b) Insert values into the table

i) Insert tuples into Publishers

```
INSERT INTO Publishers (publisher_id, publisher_name) VALUES (1,  
'Pearson');  
INSERT INTO Publishers (publisher_id, publisher_name) VALUES (2,  
'McGraw-Hill');  
INSERT INTO Publishers (publisher_id, publisher_name) VALUES (3,  
'Oxford');
```

```
INSERT INTO Publishers (publisher_id, publisher_name) VALUES (4, 'Cambridge');
```

ii) Insert tuples into Students

```
INSERT INTO Students (student_id, first_name, last_name, dob, email)
VALUES (1, 'John', 'Doe', '2000-05-15', 'john.doe@example.com');
INSERT INTO Students (student_id, first_name, last_name, dob, email)
VALUES (2, 'Jane', 'Smith', '2001-09-25', 'jane.smith@example.com');
INSERT INTO Students (student_id, first_name, last_name, dob, email)
VALUES (3, 'Emily', 'Johnson', '2000-11-10', 'emily.johnson@example.com');
```

 RAMAIAH
College of Arts, Science
& Commerce - Autonomous

iii) Insert tuples into Books

```
INSERT INTO Books (book_id, title, author, price, publisher_id) VALUES
(1, 'Database Systems', 'Korth', 250.00, 1);
INSERT INTO Books (book_id, title, author, price, publisher_id) VALUES
(2, 'Operating Systems', 'Silberschatz', 300.00, 2);
INSERT INTO Books (book_id, title, author, price, publisher_id) VALUES
(3, 'Artificial Intelligence', 'Russell', 450.00, 3);
INSERT INTO Books (book_id, title, author, price, publisher_id) VALUES
(4, 'Computer Networks', 'Tanenbaum', 180.00, 4);
```

iv) Insert tuples into Book_Issues

```
INSERT INTO Book_Issues (issue_id, student_id, book_id, issue_date,
return_date) VALUES (1, 1, 1, '2025-04-01', '2025-04-15');
INSERT INTO Book_Issues (issue_id, student_id, book_id, issue_date,
return_date) VALUES (2, 2, 2, '2025-04-02', '2025-04-16');
INSERT INTO Book_Issues (issue_id, student_id, book_id, issue_date,
return_date) VALUES (3, 3, 4, '2025-04-03', '2025-04-17');
```

c) Display different publishers from the lists

```
SELECT DISTINCT publisher_name FROM Publishers;
```

d) List details of all the books whose price range between 100 and 300

```
SELECT * FROM Books WHERE price BETWEEN 100 AND 300;
```

e) List Books Published by a Specific Publisher

```
SELECT B.title, B.author, B.price  
FROM Books B  
JOIN Publishers P ON B.publisher_id = P.publisher_id  
WHERE P.publisher_name = 'Pearson';
```

f) List All Book Titles with the Corresponding Student's First and Last Names and the Issue Date

```
SELECT B.title, S.first_name, S.last_name, BI.issue_date  
FROM Book_Issues BI  
JOIN Books B ON BI.book_id = B.book_id  
JOIN Students S ON BI.student_id = S.student_id;
```

g) List All Memberships with Student Name, Start Date, and End Date

```
SELECT S.first_name, S.last_name, M.start_date, M.end_date  
FROM Memberships M  
JOIN Students S ON M.student_id = S.student_id;
```

h) Find Students Who Have Not Returned Books After the Due Date

```
SELECT S.first_name, S.last_name, B.title, BI.issue_date, BI.return_date
FROM Book_Issues BI
JOIN Students S ON BI.student_id = S.student_id
JOIN Books B ON BI.book_id = B.book_id
WHERE (BI.return_date IS NULL OR BI.return_date > SYSDATE)
AND BI.issue_date < SYSDATE;
```

**RAMAIAH**College of Arts, Science
& Commerce - Autonomous

6. Case Study: Employee Payroll Management System

Objective: To design and implement a database schema for managing employee payroll information. This involves handling data for employees, their departments, pay details, and payroll records, and performing various SQL operations to retrieve and analyze data.

Employees – employee_id INT PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50), dob DATE, hire_date DATE, department_id INT (foreign key), position VARCHAR(50), email

Departments – Department_id as primary key and dept_name varchar(20)

PayDetails – pay_detail_id INT PRIMARY KEY, employee_id INT (Foreign key), basic_salary DECIMAL(10, 2), bonuses DECIMAL(10, 2), deductions DECIMAL(10, 2),

PayrollRecords – payroll_id INT PRIMARY KEY, employee_id INT (foreign key), pay_period_start DATE, pay_period_end DATE, gross_salary DECIMAL(10, 2), net_salary DECIMAL(10, 2), payment_date DATE

- a) Create the above tables.
- b) Insert 5 tuples into each tables.
- c) List All Employees with Their Department Names
- d) List Employees and Their Pay Details.
- e) Total Payroll Expense for Each Department.
- f) List of Employees with Their Last Payroll Record.
- g) List All Employees Who Have Not Received Their Payroll for the Current Month
- h) Calculate the Average Gross Salary for Employees in Each Department

- a) Create the above tables.

i) CREATE TABLE Departments (
department_id INT PRIMARY KEY,
department_name VARCHAR(100)
);

ii) CREATE TABLE Employees (

employee_id INT PRIMARY KEY,

first_name VARCHAR(50),

last_name VARCHAR(50),

dob DATE,

hire_date DATE,

department_id INT,

position VARCHAR(50),

email VARCHAR(100),

FOREIGN KEY (department_id) REFERENCES
Departments(department_id)

);

iii) CREATE TABLE PayDetails (

pay_detail_id INT PRIMARY KEY,

employee_id INT,

basic_salary DECIMAL(10, 2),

bonuses DECIMAL(10, 2),

deductions DECIMAL(10, 2),

foreign_key (employee_id) REFERENCES Employees(employee_id)

);

```
iv) CREATE TABLE PayrollRecords (  
    payroll_id INT PRIMARY KEY,  
    employee_id INT,  
    pay_period_start DATE,  
    pay_period_end DATE,  
    gross_salary DECIMAL(10, 2),  
    net_salary DECIMAL(10, 2),  
    payment_date DATE,  
    FOREIGN KEY (employee_id) REFERENCES Employees(employee_id)  
);
```

b) Insert 5 tuples into each tables.

```
i) INSERT INTO Departments (department_id, department_name) VALUES  
(1, 'HR');  
INSERT INTO Departments (department_id, department_name) VALUES (2,  
'Finance');  
INSERT INTO Departments (department_id, department_name) VALUES (3,  
'IT');  
INSERT INTO Departments (department_id, department_name) VALUES (4,  
'Marketing');
```

```
ii) INSERT INTO Employees (employee_id, first_name, last_name, dob,  
hire_date, department_id, position, email)  
VALUES (1, 'John', 'Doe', '1985-04-15', '2015-05-10', 1, 'HR Manager',  
'john.doe@example.com');
```

```
INSERT INTO Employees (employee_id, first_name, last_name, dob,
hire_date, department_id, position, email)
VALUES (2, 'Jane', 'Smith', '1990-06-30', '2018-03-20', 2, 'Accountant',
'jane.smith@example.com');
```

```
INSERT INTO Employees (employee_id, first_name, last_name, dob,
hire_date, department_id, position, email)
VALUES (3, 'Emily', 'Johnson', '1987-08-25', '2014-07-15', 3, 'Software
Engineer', 'emily.johnson@example.com');
```

```
INSERT INTO Employees (employee_id, first_name, last_name, dob,
hire_date, department_id, position, email)
VALUES (4, 'Mark', 'Williams', '1992-11-12', '2020-01-10', 4, 'Marketing
Specialist', 'mark.williams@example.com');
```

```
iii) INSERT INTO PayDetails (pay_detail_id, employee_id, basic_salary,
bonuses, deductions)
VALUES (1, 1, 5000.00, 500.00, 100.00);
INSERT INTO PayDetails (pay_detail_id, employee_id, basic_salary,
bonuses, deductions)
VALUES (2, 2, 4000.00, 300.00, 50.00);
INSERT INTO PayDetails (pay_detail_id, employee_id, basic_salary,
bonuses, deductions)
VALUES (3, 3, 7000.00, 1000.00, 200.00);
INSERT INTO PayDetails (pay_detail_id, employee_id, basic_salary,
bonuses, deductions)
VALUES (4, 4, 3500.00, 200.00, 30.00);
```

```
iv) INSERT INTO PayrollRecords (payroll_id, employee_id,
pay_period_start, pay_period_end, gross_salary, net_salary, payment_date)
VALUES (1, 1, '2025-03-01', '2025-03-31', 5400.00, 5300.00, '2025-04-05');
```

```
INSERT INTO PayrollRecords (payroll_id, employee_id, pay_period_start,
pay_period_end, gross_salary, net_salary, payment_date)
VALUES (2, 2, '2025-03-01', '2025-03-31', 4300.00, 4200.00, '2025-04-05');
```

```
INSERT INTO PayrollRecords (payroll_id, employee_id, pay_period_start,
pay_period_end, gross_salary, net_salary, payment_date)
VALUES (3, 3, '2025-03-01', '2025-03-31', 8000.00, 7800.00, '2025-04-05');
```



```
INSERT INTO PayrollRecords (payroll_id, employee_id, pay_period_start,  
pay_period_end, gross_salary, net_salary, payment_date)  
VALUES (4, 4, '2025-03-01', '2025-03-31', 3700.00, 3670.00, '2025-04-05');
```

c) List All Employees with Their Department Names

```
SELECT E.first_name, E.last_name, D.department_name  
FROM Employees E  
JOIN Departments D ON E.department_id = D.department_id;
```

d) List Employees and Their Pay Details

```
SELECT E.first_name, E.last_name, P.basic_salary, P.bonuses, P.deductions  
FROM Employees E  
JOIN PayDetails P ON E.employee_id = P.employee_id;
```

e) Total Payroll Expense for Each Department

```
SELECT D.department_name, SUM(PR.net_salary) AS  
total_payroll_expense  
FROM PayrollRecords PR  
JOIN Employees E ON PR.employee_id = E.employee_id  
JOIN Departments D ON E.department_id = D.department_id  
GROUP BY D.department_name;
```

f) List of Employees with Their Last Payroll Record

```
SELECT E.first_name, E.last_name, PR.pay_period_start,  
PR.pay_period_end, PR.net_salary  
FROM Employees E  
JOIN PayrollRecords PR ON E.employee_id = PR.employee_id  
WHERE PR.payroll_id IN (  
    SELECT MAX(payroll_id)  
    FROM PayrollRecords  
    GROUP BY employee_id  
);
```

g) Calculate the Average Gross Salary for Employees in Each Department

```
SELECT D.department_name, AVG(PR.gross_salary) AS avg_gross_salary
FROM PayrollRecords PR
JOIN Employees E ON PR.employee_id = E.employee_id
JOIN Departments D ON E.department_id = D.department_id
GROUP BY D.department_name;
```

h) List All Employees Who Have Not Received Their Payroll for the Current Month

```
SELECT E.first_name, E.last_name, PR.pay_period_start,
PR.pay_period_end, PR.payment_date
FROM Employees E
JOIN PayrollRecords PR ON E.employee_id = PR.employee_id
WHERE (PR.payment_date IS NULL OR MONTH(PR.payment_date) <>
MONTH(GETDATE()))
AND YEAR(PR.payment_date) = YEAR(GETDATE());
```